

Overview

Welcome to Semantica

Core Concepts

Core Concepts

Modules

Choose the Right Module

Glossary

Guides

Architecture

Learning More

GraphRAG: Graph-Augmented Retrieval

Ingest Anything

LLM Integrations

Pipeline Builder

Multi-Agent Systems

Decision Intelligence

Context Graphs

Agent Memory

Provenance & Audit Trails

Semantic Extraction

Ontology Management

Reasoning & Rules

Export & Serialization

Deduplication & Entity Merging

MCP Server

SHACL Validation

Conflict Detection & Resolution

Change Management & Versioning

Policy Engine

Visualization

Distance Intelligence

Graph Analytics

Integrations

Agno Integration

CrewAI Integration

LangChain Integration

Docling Integration

Snowflake Integration

Databricks Integration

Salesforce Integration

Graph Stores

Apache AGE Graph Store

Vector Stores

pgvector Store

FAQ

FAQ

Community

Community

Community Projects

Contributing

Governance

Citation

License

Core Concepts

Choose the Right Module

Map your goal to the right Semantica module in under 30 seconds.

Every module works independently: import only what you need. This page maps developer goals to starting points. The [Module Reference](#) covers every module in depth.

Quick Reference

Find your goal below. The **Module** column is your import path; **Key class** is what you instantiate first.

I WANT TO...	MODULE	KEY CLASS
Load a PDF, DOCX, HTML, CSV, or archive	ingest	FileIngestor
Crawl a website	ingest	WebIngestor
Load Parquet files or partitioned datasets	ingest	ParquetIngestor
Ingest XML with schema validation	ingest	XMLIngestor
Ingest from SQL, Snowflake, Databricks, Kafka, or email	ingest	DBIngestor , SnowflakeIngestor , DatabricksIngestor , StreamIngestor
Extract clean text and tables from a document	parse	DocumentParser
Parse complex PDFs with OCR or multi-column layout	parse	DoclingParser
Chunk text for embedding or RAG	split	TextSplitter
Normalize text, dates, entities, or encodings	normalize	TextNormalizer , EntityNormalizer
Find named entities (people, orgs, locations) in text	semantic_extract	NERExtractor
Extract typed relationships from text	semantic_extract	RelationExtractor
Extract RDF subject–predicate–object triplets	semantic_extract	TripletExtractor
Build a queryable knowledge graph	kg	GraphBuilder
Add time-validity (valid_from / valid_until) to facts	kg	TemporalGraphQuery
Run graph algorithms (centrality, communities, paths)	kg	GraphAnalyzer , CentralityCalculator
Generate vector embeddings	embeddings	EmbeddingGenerator
Store and search vectors	vector_store	VectorStore
Persist a graph in Neo4j or FalkorDB	graph_store	Neo4jStore , FalkorDBStore
Store RDF triples and query with SPARQL	triplet_store	TripletStore
Deduplicate entities across sources	deduplication	DuplicateDetector , EntityMerger
Detect and resolve contradictory facts	conflicts	ConflictDetector , ConflictResolver
Give an AI agent persistent memory	context	AgentContext
Ground LLM responses in a knowledge graph (GraphRAG)	context	AgentContext.query_with_reasoning()
Record AI decisions with a full audit trail	context	AgentContext.record_decision()
Search past decisions before making a new one	context	AgentContext.find_precedents()
Trace the causal chain of a decision	context	AgentContext.get_causal_chain()
Track where every fact came from (W3C PROV-O)	provenance	ProvenanceManager
Version a graph with checksums and rollback	change_management	TemporalVersionManager
Auto-generate an OWL schema from a graph	ontology	OntologyGenerator
Validate a graph against SHACL constraints	ontology	SHACLGenerator , OntologyValidator
Derive new facts from existing knowledge	reasoning	Reasoner , GraphReasoner
Export to RDF Turtle, JSON-LD, or N-Triples	export	RDFExporter
Export to Parquet for Spark / BigQuery	export	ParquetExporter
Export for ArangoDB	export	ArangoAQLExporter
Export to Neo4j or Memgraph via Cypher	export	LPGEExporter
Visualize a knowledge graph interactively	visualization	KGVisualizer
Run a reproducible multi-step pipeline	pipeline	PipelineBuilder
Use Semantica from Claude Desktop or Cursor	mcp_server	semantica-mcp
Bootstrap a graph from verified seed data	seed	SeedDataManager
Extend Semantica with a custom component	core	PluginRegistry

Goal-by-Goal Starting Points

Pick your goal to see the minimum imports and a working skeleton.

Build a Knowledge GraphBuild GraphRAGAdd Agent MemoryTrack ProvenanceExport

Turn documents, web pages, or databases into a structured, queryable graph.

Pipeline: ingest → parse → semantic_extract → kg

```
from semantica.ingest import FileIngestor
from semantica.parse import DocumentParser
from semantica.semantic_extract import NERExtractor, RelationExtractor
from semantica.kg import GraphBuilder

sources = FileIngestor().ingest("report.pdf")
parsed = DocumentParser().parse_document("report.pdf")

# No API key required: pattern-based extraction
entities = NERExtractor(method="pattern").extract(parsed)
relationships = RelationExtractor(method="rule").extract(parsed, entities=entities)

graph = GraphBuilder(merge_entities=True).build(
    sources=[{"entities": entities, "relationships": relationships}]
)
print(f"{len(graph.nodes)} nodes, {len(graph.edges)} edges")
```

Pass method="pattern" to NERExtractor for zero-cost, zero-API-key extraction. Switch to method="llm" with any of the supported providers for higher recall.

See the [Quickstart →](#) for a full pipeline with visualization and export.

Architecture Selection Guidance

- Knowledge graph vs. vector store selection
- Fast local pipeline setup
- Minimum configuration for existing agents
- Minimum stack for compliance-ready pipelines

- Quickstart:** full pipeline in 5 minutes.
- Module Reference:** every module with examples and common chains.
- API Reference:** complete class and method documentation.

< Modules

Glossary >

